



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations



BCSE204L- Design and Analysis of Algorithms- Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	Design Paradigms: Greedy, Divide and Conquer Techniques	6 hours
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	String Matching Algorithms	5 hours
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	Graph Algorithms	6 hours
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	Geometric Algorithms	4 hours
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	Randomized algorithms	5 hours
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	Classes of Complexity and Approximation Algorithms	7 hours
<u>The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman</u>		
Module:8	Contemporary Issues	2 hours



BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos ,Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K.Ahuja, Thomas L.Magnanti and James B.Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.

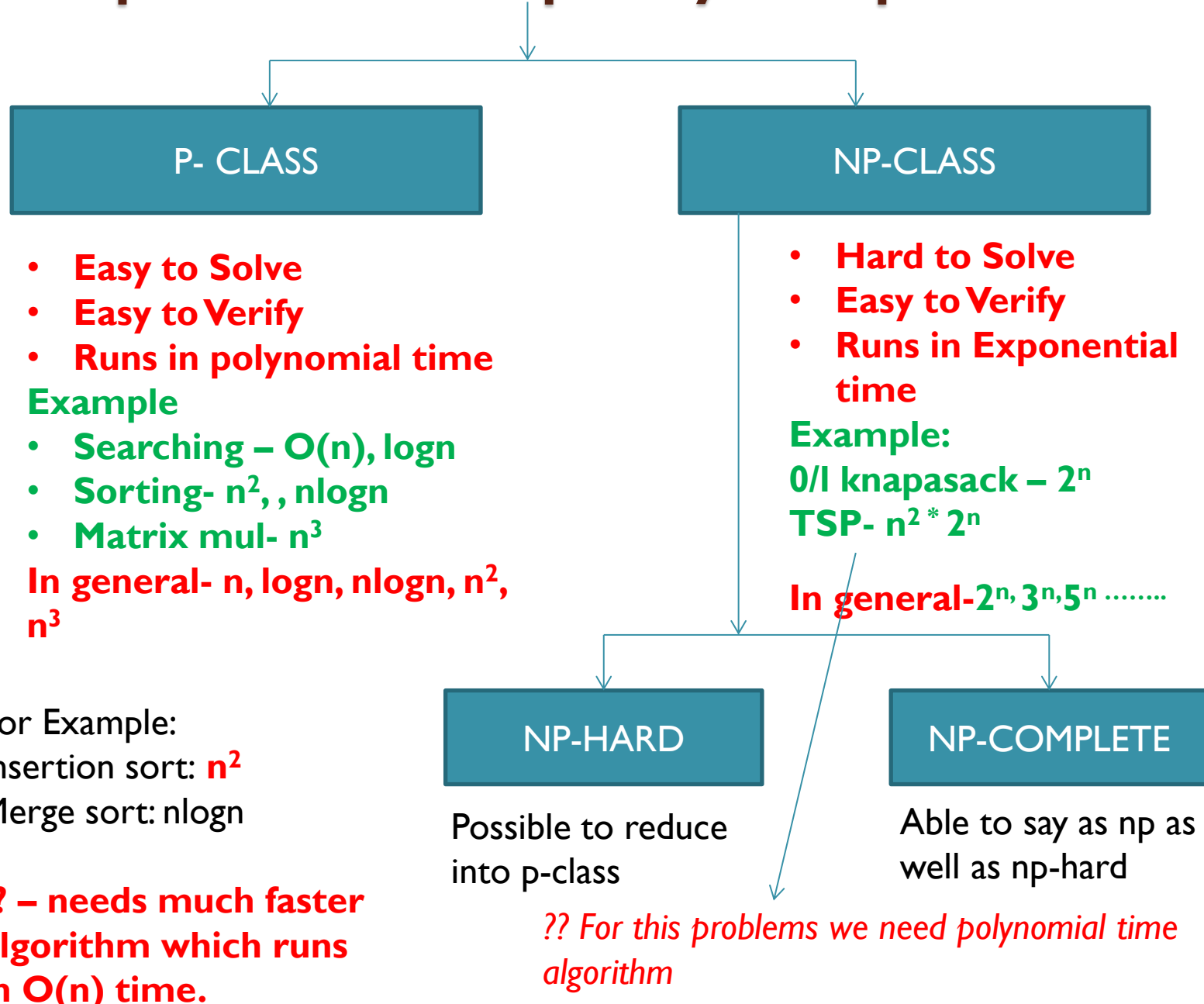


Module 7 : Classes of Complexity and Approximation Algorithms (7 Hours)

- **Class P- NP- Reducibility and NP- Completeness**
- **SAT, 3-SAT**
- **Independent Set**
- **Clique Decision Problem**
- **Approximation Algorithm**
 - **Vertex Cover**
 - **Set Cover**
 - **Travelling Salesman Problem**

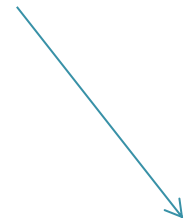


Computational Complexity – Requirement??



Computational Complexity – Requirement??

- No Solution so far for converting exponential time taking algorithm to polynomial time taking algorithm.
- **So what's next??**
- Something we have to make it as useful
- Guidelines/Framework were made to do research on those problems which are running in exponential time algorithm.



NP Hard and NP- Complete





For the exponential time algorithm when we are not able to provide polynomial time solutions

- Try to Find any similarities between those problems.
- So that, if one problem is solved, other also can be done. (either solve or relate with some other)
- When we are unable to write deterministic algorithm? - Why cant we try Non-Deterministic

Once we don't know to how it is working

From Magical Algorithm (Non Deterministic)

Once we know

Technique /Procedure (Deterministic Algorithm)

i.e. When we are unable to write deterministic polynomial time algorithm? - Why cant we try Non-Deterministic polynomial time algorithm

NP-Hard and NP-Complete

Example Non Deterministic Algorithm

Algorithm Nsearch(A,n,Key)

```
{
  j=choice();
  if key=a[j];
  {
    write(j);
    success();
  }
  Write(0);
  Failure();
}
```

$O(1)$

$O(1)$

$O(1)$

*We don't know
how it is working*

*Gives index of the
key element*

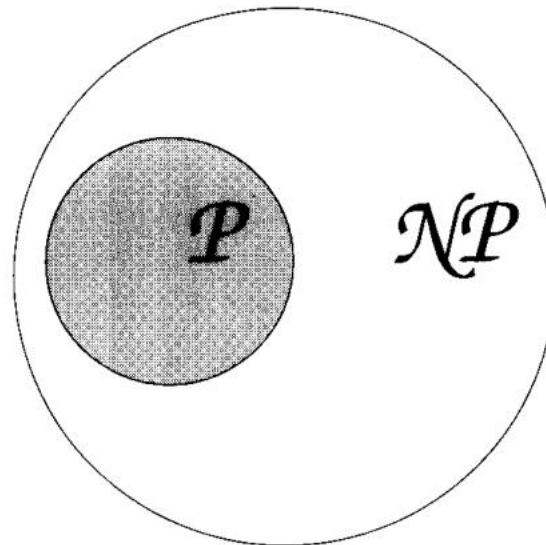
*Note: Like this we can write
the non-deterministic
polynomial time algorithm
for problems which takes
exponential time.*

Non-deterministic statements



P & NP classes - Relationship

- **P** is the set of all decision problems solvable by Deterministic algorithms in polynomial time.
- **NP** is the set of all decision problems solvable by nondeterministic algorithms in polynomial time.
- Relationship between P and NP.



P is subset of NP
i.e. the deterministic algorithm that we know today, where it is part of non-deterministic algorithm yesterday



Base Problems related with exponential time taking algorithm(Guideline 2)

SATISFIABILITY

- Let $\mathbf{X_i = (X_1, X_2, X_3)}$ denote boolean variables (their value is either true or false).
- A literal is either a variable or its negation.
- A formula in the propositional calculus is an expression that can be constructed using literals and the operations **AND, OR**.
- CNF- Conjunctive Normal Form/ Propositional calculus formula
- What is Satisfiability Problem??

The satisfiability problem is to determine whether a formula is true for some assignment of truth values to the variables.

- CNF-satisfiability is the satisfiability problem for CNF formulas.



CNF- Satisfiability

CNF – Propositional Calculus Formula

$$\bullet \text{ CNF} = (X_1 \vee \overline{X_2} \vee X_3) \Delta (\overline{X_1} \vee X_2 \vee \overline{X_3})$$

Clause C1

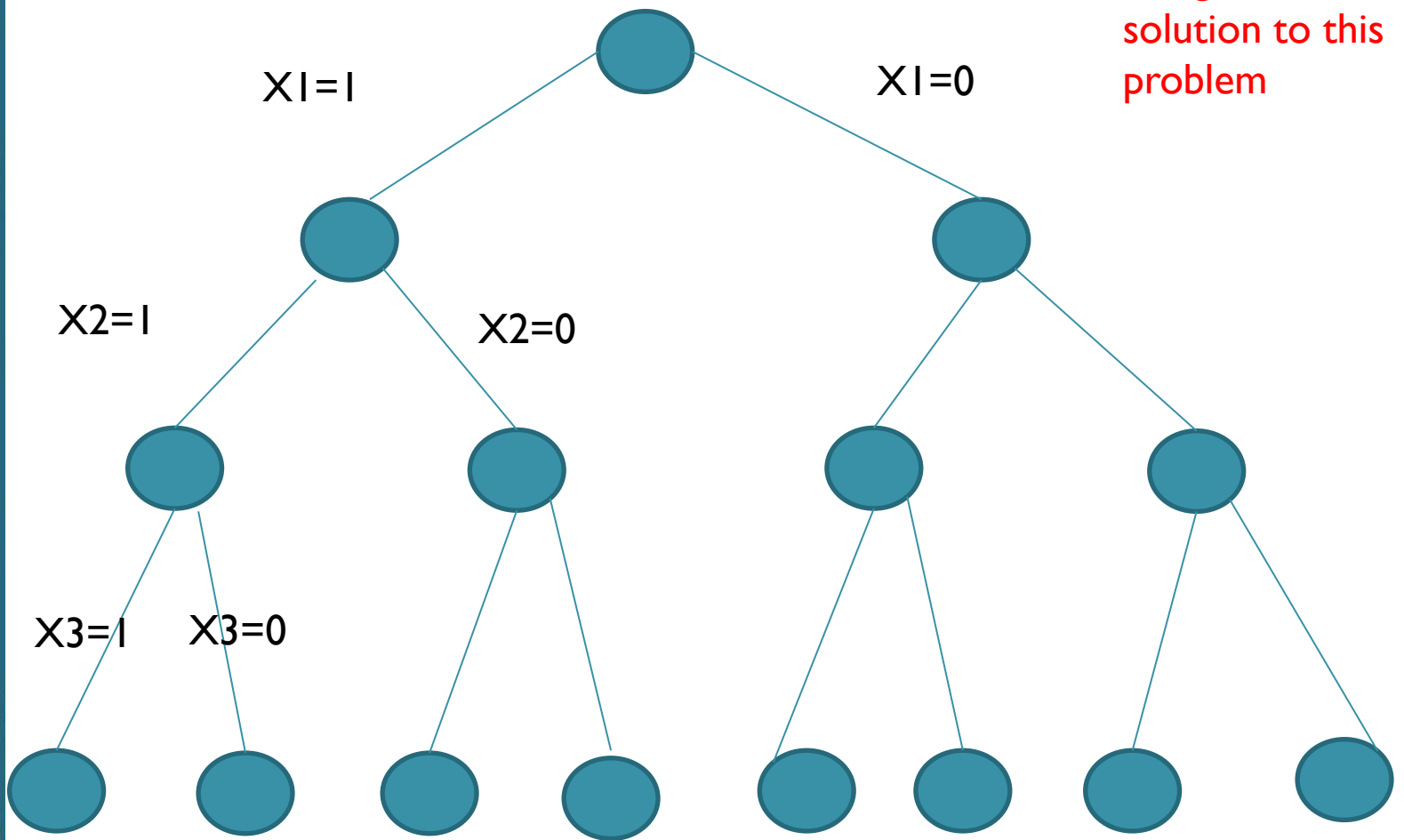
Clause C2

- For what values of x the above formula is true??
- What is the time taken??
- 2^3
- If n inputs, 2^n - which is considered as exponential time algorithm.

x1	x2	x3
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

How this problem is represented??

Satisfiability- State Space Tree



Path from root to leaf gives the solution to this problem

Show this problem is similar to 0/1 knapsack, TSP, SOS, GC, HC etc.....

What is the understanding here??

- If Satisfiability is solved in polynomial time, *others(0/1knapsack, TSP, HC, GC, SoS)* can also be solved in polynomial time
- Or any one of them is solved others can also be solved.
- ***How to prove it??***



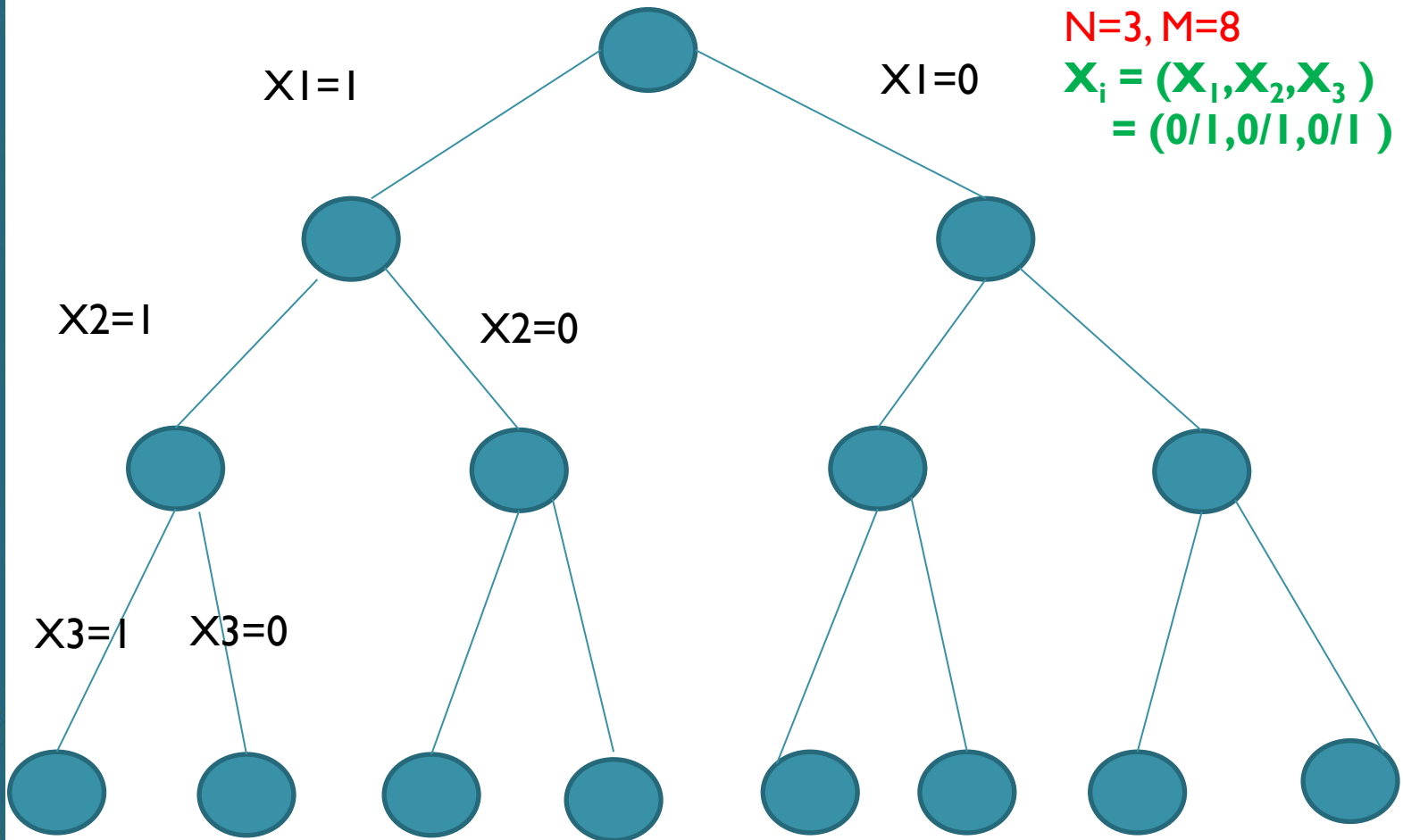
Satisfiability- 0/1 Knapsack

$P = \{10, 8, 12\}$

$W = \{5, 4, 3\}$

$N = 3, M = 8$

$X_i = (X_1, X_2, X_3)$
 $= (0/1, 0/1, 0/1)$



NP-HARD

- **Exponential time taking algorithm**

0/1 Knapsack

Travelling Salesman Problem

Sum of Subsets

Graph Coloring

Hamiltonian Cycles

2^n

HARD
PROBLEMS- Taking
Exponential time

- **Satisfiability - 2^n**

- Taken as Base Problem – so called as Np-Hard

- **How to relate?? - Reduction**



Reducibility

- **What is reducibility??**
- Let L_1 and L_2 be problems. Problem L_1 reduces to L_2 (also written $L_1 \leq L_2$) if and only if there is a way to solve L_1 by a deterministic polynomial time algorithm using a deterministic algorithm that solves L_2 in polynomial time.
- **Transitive relation**
- The above definition implies that if we have a polynomial time algorithm for L_2 , then we can solve L_1 in Polynomial time.
- We can verify that if $(L_1 \leq L_2)$ and $(L_2 \leq L_3)$ then $L_1 \leq L_3$



Reducibility

- **SAT α 0/1 Knapsack** means satisfiability reduces to 0/1 knapsack problem.
- NP-Hard α NP-Hard (Some instances of Satisfiability(i.e One Formula “ I_1 ”) converted into 0/1 knapsack problem(“ I_2 ”))
- **Observation:**
- If Satisfiability reduces to any problem- That problem also becomes NP-Hard.
- This is showing relations among problems



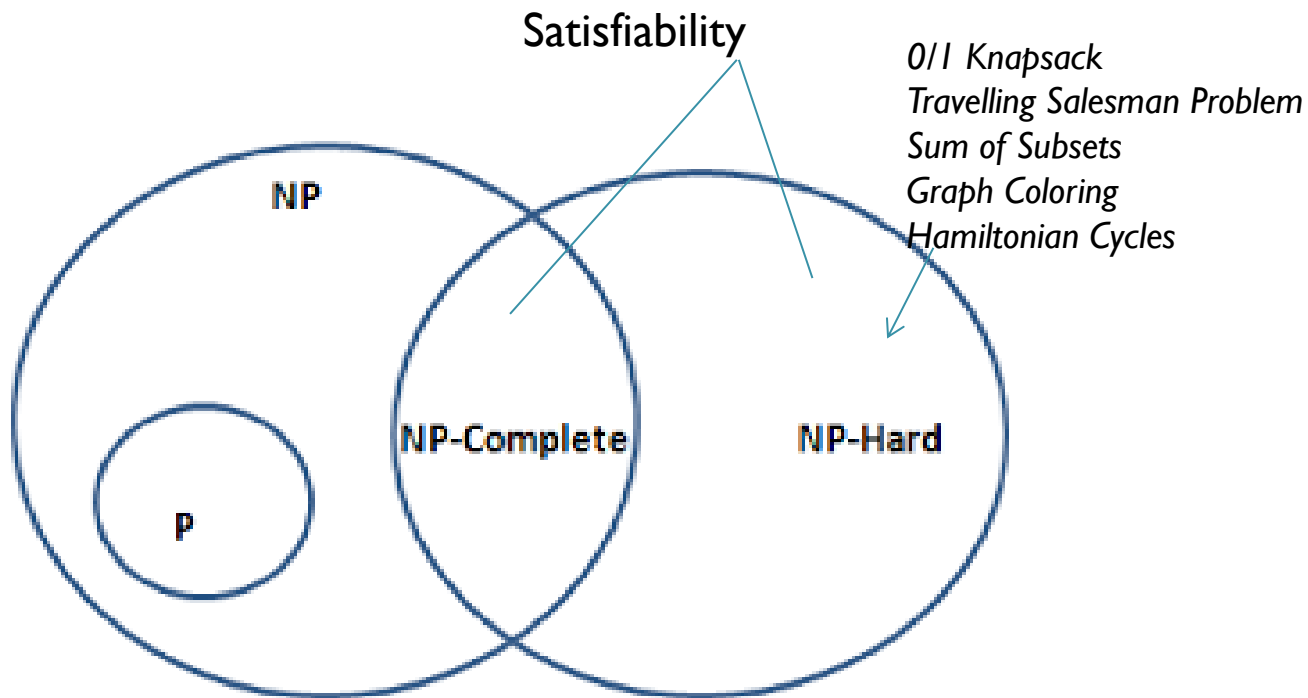
Writing Non- Deterministic Algorithms

- One more thing we do is **Writing Non-deterministic Algorithm**
- Do we have any non-deterministic algorithm for any of this problem?
- **YES - SAT**
- It is having Non-Deterministic Polynomial Time Algorithm- It is existing.
- So SAT is a known as NP-Complete Problem
- SAT is **NP-Hard as well Np-Complete problem** – because it is having existing non-deterministic polynomial time algorithm.
- SAT- Known **Np-Complete problem**
- **SAT α L** - If we write non-deterministic polynomial time algorithm it becomes NP-Complete.

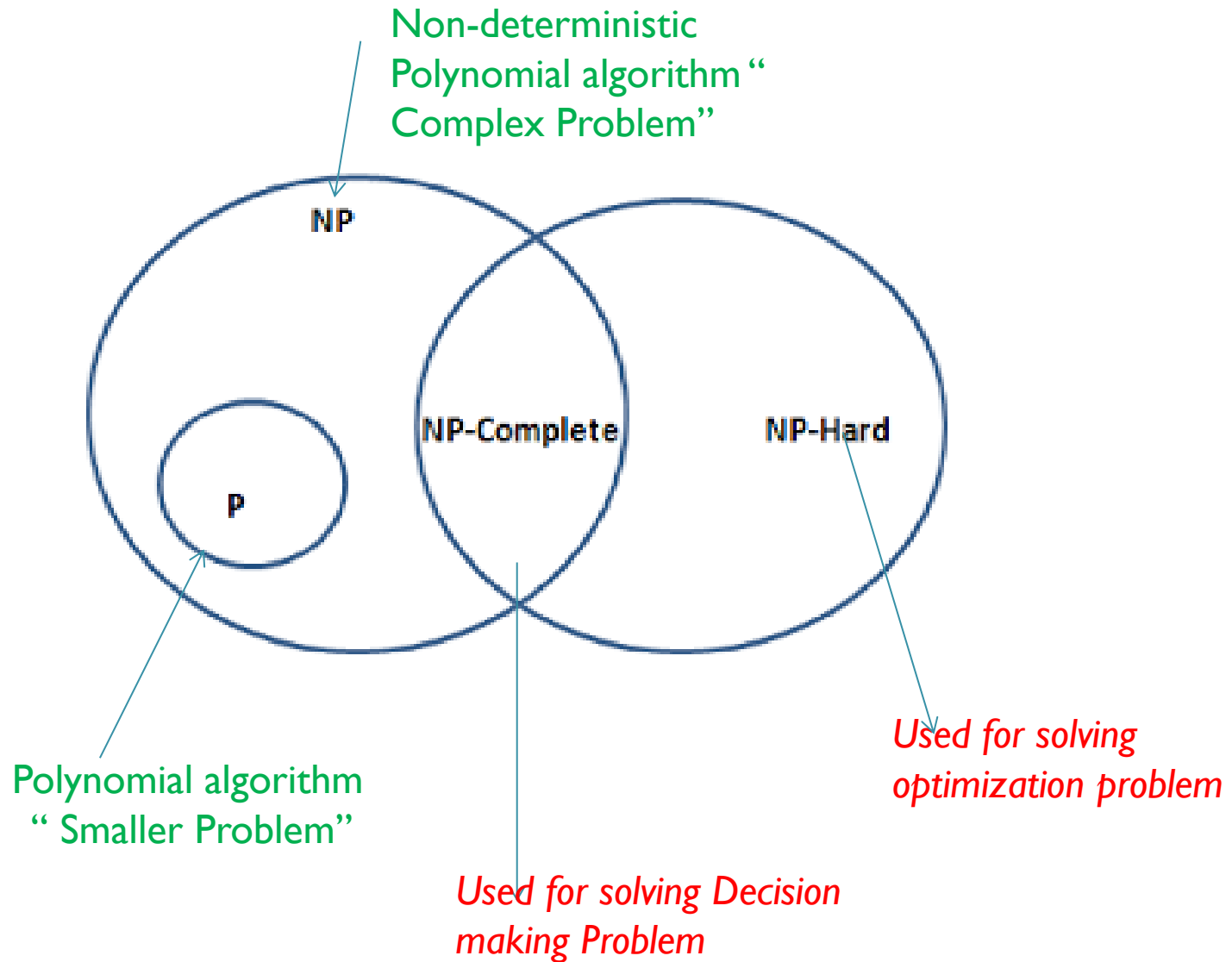


NP-HARD & NP-Complete

- A Problem L is NP-Hard if and only-if satisfiability reduces to L (Satisfiability α L).
- A Problem L is NP-Complete if and only if L is NP-Hard and L belongs to NP.



Representation- Computational Complexity



- There are Np-Hard Problems that are Not Np-Complete



NP-HARD & Np- Complete

- Since deterministic algorithms are just a special case of nondeterministic ones, we conclude that $P \subseteq NP$.
- What we do not know, and what has become perhaps the most famous unsolved problem in computer science, is whether $P = NP$ or $P \neq NP$.
- Is it possible that for all the problems in NP, there exist **polynomial time deterministic algorithms** that is remained undiscovered? **YES**
- Cook formulated the following question: Is there any single problem in NP such that if we showed it to be in P, then that would imply that $P = NP$?
- Cook Theorem – Satisfiability is in P if and only if $P = NP$

